

CANDY DIALOGUE ENGINE

Live Cutsscenes

1.1.0

Live cutsscenes are cutsscenes that aren't pre-recorded (as opposed to video cutsscenes). During live cutsscenes, code causes characters to move, environments to load, camera angles to change, sounds to play or speech text to be displayed. Synchronizing all of this can get complicated.

Luckily, Candy DE doesn't just let you display dialogue during live cutsscenes: it goes further and provides commands that can control game events in perfect timing with dialogue.

For example, you can feature some spoken lines, then insert command lines to animate some characters on screen, wait for the animations to finish, and finally add more speech lines.

These commands are called §CS commands (short for 'Cutscene') and are used to interact with nodes directly.

Basics

§CS commands adapt to your scene and node structure. You don't have to setup scenes in any specific way dictated by Candy DE.

To make things easier for yourself, you'll want to group nodes under common parents. For example, NPCs might be children of a Node named "Characters", and cameras might be children of a Node named "Cameras".

§CS Commands work by first locating these parent nodes, and then finding the specific child node a command must interact with.

The paths to these parent nodes should be stored in the **cs_locations** dictionary in the **Candy_Properties.gd** script:

```
#^ Cutscene Commands paths:
#TODO: Edit paths to fit your game structure, add path keys as needed.
var cs_locations = {
    "Player": "/root/Game/Actors",
    "Actors": "/root/Game/Actors",
    "Vehicles": "",
    "Objects": "",
    "Cameras": "/root/Game/Cameras",
    "Markers": "/root/Game/Pins",
    "WP": "/root/Game/Waypoints",
    "Lights": "/root/Game/Lights",
}
```

You can add as many parent nodes (keys) as you want to the dictionary, and you don't need to follow the provided template.

For example, you could have multiple parent nodes/paths for actors: one for exploration mode and one for combat mode. Or you could group actors, vehicles and objects under the same parent.

What you should avoid is having unpredictable node paths in your scenes, such that when adding §CS commands to your dialogues, you don't know where the nodes will be located.

If you must have unpredictable paths, you'll have to find workarounds such as dynamically updating the keys in the `cs_locations` dictionary or even editing the command line data in the dialogue scripts at runtime. The simplest option is probably to provide variable references to §CS commands instead of keys from the `cs_locations` dictionary (see the Variables guide to learn how to provide variable references to dialogue lines in place of actual values).

Once you have your node paths setup in the `cs_locations` dictionary, you can start adding §CS commands to your dialogues.

Also, note that you can provide §CS commands with a direct node paths instead of keys. See the Variables guide for details.

Lastly, remember that nodes that will be targeted by §CS commands should have predictable names. Nodes that represent characters should be named the same as actor references in the **actors** dictionary (in the **Candy_Database.gd** script).

§CS Commands

The following §CS commands are available:

§CS_Scene

This command instantiates a scene as a child of a target node.

Scene → The path to the scene(s) to instantiate, in your project's resource folders.

Path 1 → The path to the node that will be the parent of the scene.

Targets → The names of the nodes that the scene will be instantiated under.

Name → The name(s) to assign to scene instances.

Scene paths should be written in the "res://" format.

Multiple scene instances can be created: provide multiple scenes in the "Scene" field, separated with a comma (,) (e.g. "res://scene_1, res://scene_2"). To instantiate the same scene multiple times, write the scene as many times as you want instances of it.

Multiple names can be provided in the "Name" field, separated by a comma (,). Names are matched to scene instances in order (the first name is assigned to the first instance, the second name is assigned to the second instance, etc.). If there are more scene instances than names, the excess

instances are not renamed. To skip the assignment of a name to an instance, use an empty space followed by a comma in the "Name field", for example:

```
Scene = "res://scene_1, res://scene_2, res://scene_3"  
Name = "name_1, , name_2"  
→ scene_1 = name_1  
→ scene_2 = no rename  
→ scene_3 = name_2
```

Targets must be separated with a comma (,).

You can provide role references as targets, in which case Candy DE will automatically replace them with their matching actor reference.

§CS_Loc

This command moves (teleports) a target node to the position of another node (marker). It can optionally also rotate the target node to match the marker's rotation.

Path 1 → The path to the target node.

Target → The name of the node that will be moved and/or rotated.

Path 2 → The path to the marker node.

Marker → The node that the target is moved to.

Rotate → Whether to rotate the target to match the marker.

This command can be great for instantly moving cameras or lights in a scene, or re-arranging actors and objects. Small tip: add a fade to black §Effect before §CS_Loc to hide the teleportation.

§CS_Look

This command rotates target nodes to face toward marker nodes.

Path 1 → The path to the target nodes.

Target → The name of the nodes that will be rotated.

Path 2 → The path to the marker nodes.

Marker → The nodes that the targets will face.

Axis → The axis that is treated as the 'front' of the targets.

If a single marker is provided, all targets face towards it. If multiple markers are provided, targets and markers are paired (in the same order they are listed) and targets face toward their own marker (if there are more targets than markers, the extra targets are ignored).

The axis designates the 'front' of the targets. In Godot, the default is usually -Z but it ultimately depends how your meshes or sprites are oriented. As an example, if the front of a character's mesh is the +Z axis, you would set the command's axis to +Z.

Alternately, you can modify the axis value to make targets turn their back to a marker, or to make targets turn sideways relative to a marker (for example, if you want several characters to stand shoulder to shoulder).

§CS_Move

It seems obvious that there should be a command to set nodes (such as characters, vehicles or machines) in motion.

Unfortunately, there are too many unknown factors to create such a universal, ready-made command: some games use Godot's navigation server; others write custom pathfinding or AI managers. Some use physics or feature collision-avoidance logic, some don't.

As a compromise, Candy DE provides the §CS_Move command, which is empty by default. You can write code for it in [Candy_Engine.gd](#) to fit any method you want. You can read the Adding Commands guide to help you, and you can use the other commands as basic templates; if you can implement movement logic in your game outside of Candy DE, then writing your own §CS_Move command should be moderately easy.

The Candy Dialogue Creator supports the §CS_Move command, with many data fields: you don't have to use all of them, and you don't need to use them for data that matches their names either. If you want to customize the §CS_Move command in the Candy Dialogue Creator, instructions are provided in the Adding Commands guide.

§CS_Light

This command can modify the energy and/or the color of a Light2D or Light3D node. This includes the various sub-nodes: DirectionalLight, OmniLight and SpotLight.

Path 1 → The path to the target light node.

Target → The name of the light node that will be modified.

Operator → Whether to set the light energy to a fixed value, or increase, decrease, multiple or divide the energy by a value.

Energy → The energy value.

Color → The color to set the light to.

§CS_Cam

This command changes the active camera. This is fairly simple for now, but in the future it might be updated to optionally handle more camera controls based on user feedback.

Path 1 → The path to the target camera node.

Target → The name of the camera node that will be set to active.

§CS_Visible

This command changes the visible property of a node.

Path 1 → The path to the target node's parent.

Targets → The name of the node that will have its visibility changed.

Status → What to set the node's visibility to.

For Status, 'True' enables visibility, 'False' disables it, and 'Toggle' switches it to the opposite of the current value.

§CS_Toggle

This command is meant for changing properties of your choosing on nodes. It works by calling the [candy_toggle\(\)](#) function on a target node, which you must write code for.

The [candy_toggle\(\)](#) function's code will be fairly simple, although it depends what you aim to achieve. Three values are passed as arguments: `toggle_method`, `toggle_properties`, `toggle_values`.

The idea is that `toggle_properties` will define which properties are being changed on the node, and `toggle_values` defines the value or values to change properties to/by. The last argument, `toggle_method`, can be used for additional logic.

The three values are strings, so you may want to also add code to process them, such as converting them to int, float, bool or arrays.

Path 1 → The path to the target node's parent.

Targets → The name of the node that will have its properties changed.

Method → Can define an additional logic layer.

Properties → Typically defines the node properties to modify.

Value → Typically defines the value or values to modify properties to/by.

To use the §CS_Toggle command, the target node must include the following function:

```
/* Called by the §CS_Toggle command:  
  
func candy_toggle(toggle_method, toggle_properties, toggle_value):  
    pass
```

You must then write your own logic to obtain the behavior you want, using the arguments passed to the function.

\$CS_Anim

This command is used for playing animations on nodes, through an AnimationPlayer or an AnimationTree.

Before using the command, the target nodes must have two dictionaries in their attached scripts:

```
#^ Nodes for Candy DE:
@onready var internal_node_dict = {
    "Speech_Bubble": get_node("SpeechBubble"),
    "AP 1": get_node("AnimationPlayers/1")
}
```

The internal_node_dict contains references to child nodes that Candy DE must use. In this case, it must feature at least one reference to AnimationPlayer or AnimationTree.

If you might play multiple animations at the same time on a same node, you can add references to multiple AnimationPlayer or AnimationTree nodes.

```
#^ CS Loop Data:
#? Stores loop data for animations triggered with $CSAnim
var cs_loop_data = {}
```

The cs_loop_data dictionary must be left empty. Candy DE will automatically store data there as needed.

Once you have these two dictionaries setup in your target nodes, provide the following data to the \$CS_Anim command:

Path 1 → The path to the target node's parent.

Targets → The name of the nodes that will have an animation played on.

Path 2 → The key for the desired AnimationPlayer or AnimationTree in internal_node_dict.

Animation → The name of the animation to play.

Loop → How many times the animation should play (0 = infinite).

Wait → How many loops to pause dialogue processing for.

Time → The timestamp or frame until which to pause processing for.

Play/Hold → Whether to play the animation or pause on the first frame.

Time is written in the HH:MM:SS.00 format. You can also provide a frame number, e.g. f=120.

Time and Wait are cumulative: if Wait = 3 and Time = 140, Candy DE will play the animation 3 times then wait 140 additional seconds before resuming.

If Time exceeds the length of the animation, the excess is carried over to additional loops.

If Time and Wait exceed the number of loops to play, dialogue resumes as soon as all loops have played.

The §CS_Anim command is the simplest way to make 3D characters perform gestures, make machines animate, or trigger visual effects.

§CS_Anim_Wait

This command is used to pause dialogue processing until one or more animations, triggered by §CS_Anim, have played a specific number of loops and/or reached a specific timestamp or frame.

Path 1 → The path to the target node(s) parent.

Targets → The name of the nodes to wait on.

Path 2 → The key for the AnimationPlayer or AnimationTree in internal_node_dict to wait on.

Wait → How many loops to pause dialogue processing for.

Time → The timestamp or frame until which to pause processing for.

§CS_Anim_Stop

This command is used to stop ongoing animations triggered by the §CS_Anim command.

Path 1 → The path to the target node(s) parent.

Targets → The name of the nodes to stop animations on.

Path 2 → The key for the AnimationPlayer or AnimationTree in internal_node_dict to stop.

Default → -1 = Stop on the last frame, 0 = stop on the current frame. 1+ = Pause on the given frame.

§CS_Sprite

This command is equivalent to §CS_Anim, except that it's used for making Sprite2D/3D and AnimatedSprite2D/3D nodes display an image and potentially play an animation.

Like §CS_Anim, target nodes must feature the internal_node_dict dictionary, with references to Sprite or AnimatedSprite nodes. The target nodes must also feature the cs_loop_data dictionary.

Path 1 → The path to the target node's parent.

Targets → The names of the target nodes.

Path 2 → The key for the desired Sprite or AnimatedSprite node in `internal_node_dict`.

File → The name of the image file to display (for AnimatedSprite: the Sprite Frame resource).
Leave empty to remove the file or Sprite Frame resource from the Sprite or AnimatedSprite node.

Animation → The name of the animation to play (AnimatedSprite only).

Loop → How many times the animation should play (0 = infinite).

Wait → How many loops to pause dialogue processing for.

Time → The timestamp or frame until which to pause processing for.

Play/Hold → Whether to play the animation or pause on the first frame.

For details on Loop/Wait/Time, see the `§CS_Anim` command above.

Files must be placed in **Candy_DE/Media/Characters/Sprites**, inside a sub-folder named the same as the target node.

Sprite Frame resources (.tres) must be placed in an additional sub-folder named "**Sprite_Frames**".
For example:

Candy_DE/Media/Characters/Sprites/Alice/Sprite_Frames

Multiple Targets can be specified, but only one file can be provided. In other words, all Targets must be assigned a file of the same name.

For Sprite2D or Sprite3D nodes, you'll need to use sprite sheets for animations. Sprite sheets must end with a prefix that indicates the number of rows and columns, as well as the number of empty frames in the last row, like this:

idle_4x6-2.png

In this example, 4 is the number of rows (horizontal), 6 is the number of columns (vertical) and 1 is the number of empty frames in the last row (i.e. the last row has $6 - 2 = 4$ frames).

You will also need to indicate the Frames Per Second (speed) of the animation. This is done through the `cs_sprite_fps` variable in **Candy_Properties.gd**:

```
#^ CS Sprite Default FPS:  
#? The default FPS value to use for sprites animated with the §CSSprite command.  
#? FPS = Frames Per Second, i.e. the animation speed.  
var cs_sprite_fps = 30
```

Be mindful that this is a universal variable: all `§CS_Sprite` commands will refer to it. If you need to change the FPS under particular circumstances, you can do so at runtime, either with your own code or by using the `§Set` command.

Also in **Candy_Properties.gd**, you can find **default_sprite_extension** and **default_animated_sprite_extension**:

```
#^ Default File Extensions:
#? The default extensions used by Candy DE when a media file name with no extension is provided.
#TODO: Edit extensions to match your preferences.
var default_sprite_extension = ".png"           #/ For CS_Sprite
var default_animated_sprite_extension = ".tres" #/ For CS_Sprite
```

These extensions are used by the §CS_Sprite command when the provided sprite file reference has the file extension omitted.

§CS_Sprite_Wait

This command is used to pause dialogue processing until one or more animations, triggered by §CS_Sprite, have played a specific number of loops and/or reached a specific timestamp or frame.

Path 1 → The path to the target node(s) parent.

Targets → The name of the nodes to wait on.

Path 2 → The key for the Sprite or AnimatedSprite in internal_node_dict to wait on.

Wait → How many loops to pause dialogue processing for.

Time → The timestamp or frame until which to pause processing for.

§CS_Sprite_Stop

This command is used to stop ongoing animations triggered by the §CS_Sprite command.

Path 1 → The path to the target node(s) parent.

Targets → The name of the nodes to stop animations on.

Path 2 → The key for the Sprite or AnimatedSprite in internal_node_dict to stop.

Default → -1 = Stop on the last frame, 0 = stop on the current frame. 1+ = Pause on the given frame.